

TECHNICAL_REPORT

MOEX Screener: Настольный скринер акций Московской Биржи

Технический отчет по итогам инженерной практики

Организация: ООО «ПРОКСИ» (г. Курск)

Учебное заведение: Курский государственный университет

Разработчик: Жаанбаев Б.К.

Почта: bajmurat.zaanbaev@gmail.com

Репозиторий GitHub: moex-screener

Версия продукта: v0.1.0 (MVP Release)

Технологический стек: Git | Poetry | Copier | Python 3.13 | PySide6 (Qt) | Pandas | Httpx | Loguru | Ruff | PyTest | PyTest-Qt | PyInstaller

1. Введение и анализ предметной области

1.1. Бизнес-контекст проекта

В условиях современной алгоритмической торговли и высокой динамики финансовых рынков критически важным фактором для успешной деятельности трейдера или финансового аналитика является скорость принятия решений. Специфика работы с Московской Биржей (MOEX) подразумевает обработку огромных потоков сырых рыночных данных, поступающих через открытые программные интерфейсы информационно-статистического сервера (ISS API).

Основная проблема конечного пользователя заключается в избыточности и слабой структурированности сырых пакетов данных, что делает невозможным их оперативный ручной анализ. Разрабатываемый аналитический скринер акций призван решить эту бизнес-задачу. Программный комплекс автоматизирует рутинные процессы агрегации данных, обеспечивая сквозной мониторинг, многокритериальную фильтрацию в рамках единого интерфейса.

1.2. Анализ требований к MVP

Для минимизации времени разработки продукта в рамках первой итерации были сформулированы и реализованы строгие требования к минимально жизнеспособному продукту (MVP) десктопного скринера.

Функциональные требования

- Оркестрация сетевого взаимодействия:** Автоматизированный сбор актуальных биржевых стаканов и итогов торгов по акциям через MOEX ISS API.
- Интеграция гетерогенных данных:** Реализация алгоритма объединения статических (справочных) и динамических (рыночных) структур данных по общему уникальному идентификатору финансового инструмента (тикеру/SECID) на уровне аналитического ядра.
- Десериализация и очистка данных:** Преобразование сырых пакетов JSON/XML в структурированные локальные массивы данных без потери точности.
- Многокритериальный фильтрационный движок:** Наличие панели управления для фильтрации по тикерам, ценовым диапазонам и процентным изменениям с корректной обработкой неопределенных состояний (NaN, отсутствующие данные).

- **Интерактивная визуализация:** Вывод обработанной информации в динамическую таблицу с поддержкой нативной локальной сортировки и кастомного форматирования ячеек. Возможность переключения наборов отображаемых параметров «на лету»
- **Офлайн-режим:** Наличие механизма локального резервного копирования данных в формат CSV для обеспечения работоспособности аналитического слоя при отсутствии сетевого подключения.

Нефункциональные требования

- **Отзывчивость интерфейса:** Обеспечение плавности графической оболочки. Минимизация задержек основного UI-потока при обновлении данных, поддержка стабильного частоты кадров.
- **Оптимальный объем занимаемой памяти:** Исключение нецелевого дублирования массивов данных в оперативной памяти при передаче данных из сетевого слоя в интерфейсный.
- **Отказоустойчивость и защитное программирование:** Система должна корректно обрабатывать сетевые сбои (таймауты, обрывы связи, поврежденные структуры JSON Мосбиржи) без аварийного завершения процесса.
- **Глубокая наблюдаемость:** Сквозное логирование всех этапов жизненного цикла приложения (от инициализации компонентов до сетевых сессий) с ротацией и архивацией логов для последующего аудита ошибок.
- **Критерии качества и тестируемость:** Архитектура приложения должна обладать высокой степенью модульности для обеспечения возможности изолированного модульного и интеграционного тестирования компонентов без поднятия полной среды выполнения. ПО задан жесткий технологический барьер: стабильное прохождение 100% тест-кейсов в тестовом пакете перед сборкой финального дистрибутива.

1.3. Обоснование технологического стека

Выбор технологий для проекта обусловлен требованиями к высокой производительности математических расчетов, кроссплатформенности и строгому контролю качества кодовой базы:

- **Язык программирования Python 3.11+:** Выбран в качестве базового языка разработки благодаря развитой экосистеме библиотек, поддержке современной статической типизации и высокой скорости проектирования MVP.
- **Графический фреймворк PySide6 (Qt for Python >= 6.6.0):** Промышленный стандарт для разработки нативных приложений. В отличие от альтернатив (Tkinter, CustomTkinter), PySide6 предоставляет мощную и проверенную временем архитектуру *Model-View-Controller (MVC)*, которая критически важна для плавного отображения больших табличных данных, а также предоставляет прямой доступ к аппаратным оптимизациям ядра C++.
- **Библиотека pandas (>= 2.2.0):** Вычислительное ядро системы. Использование структур данных `DataFrame` позволяет выполнять векторизованные математические операции и фильтрацию на уровне скомпилированного C-кода. Это полностью исключает медленные циклы Python при обработке рыночной информации и гарантирует высокую производительность аналитического движка.
- **Сетевой клиент HTTPX (>= 0.27.0):** Современная библиотека для выполнения HTTP-запросов. Была выбрана вместо устаревшего пакета `requests` благодаря расширенной поддержке конфигурации таймаутов, пулов соединений и улучшенной обработке низкоуровневых сетевых исключений, что повышает общую отказоустойчивость ядра.
- **Менеджер пакетов Poetry:** Инструмент детерминированного управления зависимостями. Использование механизма `poetry.lock` гарантирует стопроцентную воспроизводимость среды разработки и сборки на любой целевой машине, полностью исключая проблему «конфликта зависимостей».

2. Архитектура программного комплекса и паттерны проектирования

2.1. Многослойная архитектура проекта MVC

Программный комплекс скринера акций спроектирован по принципу многослойной архитектуры, обеспечивающей строгое разделение зон ответственности, высокую модульность и тестируемость каждого компонента. Вся кодовая база декомпозирована на четыре независимых функциональных слоя, взаимодействующих между собой сверху вниз через чистые программные интерфейсы.

- 1. Слой сетевого взаимодействия (API):** Находится в пакете `src/api/`. Его единственная ответственность - выполнение HTTP-запросов к ISS API Московской Биржи с помощью библиотеки `httpx`, управление пулами соединений, конфигурация таймаутов сессии и обработка низкоуровневых ошибок связи с выбросом кастомных исключений (`MoexAPIError`). Данный слой изолирован от интерфейса и бизнес-логики.
- 2. Слой бизнес-логики и системных компонентов (Core):** Пакет `src/core/`. Выступает центральным оркестратором данных. Модуль `parser.py` осуществляет десериализацию сырых ответов JSON и валидацию их динамических схем. Модуль `analytics.py` выполняет векторизованные математические расчеты. Также слой включает системные модули конфигурации (`config.py`) и логирования (`logger_config.py`).
- 3. Слой моделей отображения данных (Models):** Пакет `src/models/`. Адаптирует структуры данных бизнес-логики под требования графического фреймворка. Реализован классом `MoexTableModel` в файле `table_model.py`.
- 4. Слой графического интерфейса пользователя (Views):** Пакет `src/views/`. Отвечает исключительно за процесс отрисовки, визуализацию и перехват событий ввода пользователя. Компонент `filter_panel.py` агрегирует элементы управления фильтрацией, а `main_window.py` организует общую компоновку слоев и связывание сигналов интерфейса с моделью данных.

2.2. Паттерн Qt MVC (Model-View)

Отображение сложных динамических финансовых таблиц в `PySide6` требует жесткого следования архитектурному паттерну *Model-View* (нативная Qt-интерпретация паттерна MVC, где функции контроллера распределены между моделью и представлением). Вместо использования тяжеловесных стандартных контейнеров, создающих избыточные C++ обёртки для каждой ячейки, в проекте был разработан кастомный класс `MoexTableModel`, наследуемый напрямую от базового интерфейса `QAbstractTableModel`.

Технический анализ реализации `MoexTableModel` раскрывает переопределение ключевого интерфейса ядра Qt, необходимого для корректной работы `QTableView`.

Защитные условия и управление геометрий таблицы

Интерфейсные методы `rowCount()` и `columnCount()` снабжены обязательной проверкой валидности родительского индекса `if parent.isValid(): return 0`. Это исключает некорректное поведение модели в древовидных структурах и гарантирует строгое плоское табличное отображение.

Внутри метода `data()` реализовано защитное условие против состояния гонки данных:

```
if row >= len(self._df) or col >= len(self._df.columns):
    logger.warning(f"Индекс Qt [{row}, {col}] вышел за границы DataFrame ({self._df.shape}")
    return None
```

Данная проверка критически важна при асинхронном или фоновом обновлении рыночных данных. Она предотвращает аварийное завершение графического процесса (Crash), если `QTableView` запрашивает отрисовку ячейки по старому индексу в момент подмены внутренней матрицы `pandas.DataFrame`.

Контекстное разделение ролей и форматирование данных

В переопределенном методе `data()` логика полностью разделена по системным ролям `Qt.ItemDataRole`, что позволяет инкапсулировать правила форматирования внутри модели, не загрязняя слой отображения (View):

- `Qt.ItemDataRole.DisplayRole` (**Визуализация текста**): Реализована сквозная обработка неопределенных биржевых состояний. Любые null-значения (`None` или `NaN` из `pandas`) перехватываются методом `pd.isna(val)` и безопасно подменяются на константный прочерк `"—"`, исключая появление пустых или сломанных ячеек. Для валидных финансовых данных применяется строгое форматирование по группам `FORMAT_GROUPS` с локализацией чисел (замена американской запятой-разделителя тысяч на стандартный для РФ пробел через `.replace(",", " ")`):
 - **Цены** (`price_2dp`): Строго два знака после запятой (`f"{val:,.2f}"`).
 - **Доходность** (`percent`): Обязательное явное отображение математического знака для положительных и отрицательных изменений (`f"{val:+.2f}%"`).
 - **Объёмы и крупные суммы** (`integer_volume`, `large_money`): Приведение к целочисленному типу и разбиение на разряды для высокой читаемости.
- `Qt.ItemDataRole.TextAlignmentRole` (**Оптимизация финансового UX**): Реализовано автоматическое выравнивание контента в зависимости от типа данных. Текстовые идентификаторы (строковые тикеры `SECID`, краткие наименования `SHORTNAME`, коды `ISIN`) выравниваются по левому краю (`AlignLeft`). Все вычисляемые аналитические показатели, цены и объёмы принудительно выравниваются по правому краю (`AlignRight`), что является обязательным промышленным стандартом для аналитических и торговых терминалов.

Архитектура локализации заголовков (`headerData`)

Метод `headerData()` обеспечивает динамическую подмену системных технических имен колонок `DataFrame` (например, `LAST`, `VALWAVE`) на человекочитаемые русскоязычные заголовки (например, «Последняя цена», «Объем торгов») с помощью глобального словаря маппинга `COLUMN_MAPPING`. Метод спроектирован отказоустойчивым: в случае отсутствия перевода в словаре или при выходе за границы структуры данных, срабатывает деградация интерфейса (Fallback) до вывода исходного технического имени колонки, предотвращая скрытие данных.

Алгоритм быстрой локальной сортировки (`sort`)

Вместо ресурсоёмких повторных сетевых запросов или медленной сортировки средствами Qt-виджетов, метод `sort()` делегирует задачу встроенным оптимизированным C-алгоритмам `pandas`, обеспечивая мгновенный отклик интерфейса:

- **Уведомление графического конвейера**: Перед началом и после завершения манипуляций с индексами модель отправляет Qt-сигналы `layoutAboutToBeChanged` и `layoutChanged`. Это позволяет представлению заморозить отрисовку и корректно пересчитать геометрию строк.
- **Сброс сортировки** (`column == -1`): Реализовано восстановление исходного порядка строк. При передаче индекса `-1` (например, при очистке фильтров) модель вызывает `self._df.sort_index(ascending=True, inplace=True)`, возвращая таблицу к первоначальному хронологическому состоянию.
- **Обработка аномалий рынка** (`na_position="last"`): Ценные бумаги, по которым отсутствуют торги или цена не определена (`LAST == null`), при любом направлении сортировки (как по возрастанию, так и по убыванию) принудительно вытесняются в самый низ таблицы, чтобы не исказить списки

ликвидных акций. Это элегантно решено параметром `na_position="last"` в методе `.sort_values()`.

Паттерн атомарной транзакции (`set_dataframe`)

Для безопасной загрузки новых данных от API-клиента разработан изолированный метод `set_dataframe()`, реализующий паттерн транзакционного сброса состояний:

```
def set_dataframe(self, new_df: pd.DataFrame) -> None:
    self.beginResetModel()
    self._source_df = new_df.copy()
    self._df = new_df
    self.endResetModel()
```

Вызовы `beginResetModel()` и `endResetModel()` формируют неделимую транзакцию. Они принудительно уведомляют все привязанные графические виджеты (включая связанные панели фильтрации и прокрутки) о полном изменении структуры данных. В этот момент Qt сбрасывает внутренние кэши индексов, предотвращая утечки памяти и исключая рассинхронизацию UI с реальным набором строк в памяти. Метод также атомарно сохраняет эталонную мастер-копию `_source_df` для выполнения последующих операций фильтрации без разрушения исходного набора данных.

2.3. Оптимизация управления памятью

Одной из главных архитектурных проблем при разработке аналитических систем на Python является неэффективное использование CPU и оперативной памяти из-за избыточного копирования тяжелых массивов данных между слоями приложения. При наивной реализации (например, преобразовании таблицы в список списков Python `list[list]` или генерации стандартных объектов `QStandardItem` под каждую ячейку) объем занимаемой памяти приложения растет экспоненциально, а процесс десериализации блокирует GUI-поток, вызывая микрозависания (фризы) интерфейса.

В проекте эта проблема была решена внедрением **подхода с нулевым копированием** при интеграции вычислительного движка `pandas` напрямую в ядро Qt-модели:

- 1. Инкапсуляция по ссылке.** Класс `MoexTableModel` принимает в свой приватный атрибут прямую ссылку на объект `pandas.DataFrame`, который сформировал слой `Core`. Нет никакого дублирования или трансформации матрицы данных.
- 2. Прямая индексация через C-API.** Внутри переопределенного метода `data()` при обращении со стороны `QTableView` к конкретному индексу строки и столбца (`index.row()`, `index.column()`), модель обращается к ячейке `DataFrame` напрямую с использованием быстрого нативного метода целочисленной индексации `.iat[row, column]`.
- 3. Исключение оберток памяти.** Отказ от `QStandardItemModel` позволил сэкономить значительную часть памяти, поскольку Qt больше не создает тяжеловесные C++ объекты `QStandardItem` для тысяч ячеек таблицы. Все данные хранятся в непрерывных C-массивах под управлением `NumPy` внутри `pandas.DataFrame`.

Этот подход обеспечивает мгновенную ($O(1)$ по памяти) передачу данных из сетевого парсера в интерфейс. Приложение способно обрабатывать и обновлять таблицы на лету без выделения дополнительной RAM и фризов графического интерфейса, обеспечивая максимальную производительность программного комплекса на целевой машине пользователя.

3. Разработка графического интерфейса (View)

3.1. Архитектурная структура слоя представления

Слой отображения (View Layer) проекта полностью изолирован от алгоритмов бизнес-логики и построен на базе иерархической структуры нативных компонентов фреймворка `PySide6`. Основная архитектурная задача данного слоя - вывод информации на экран и трансляция пользовательских событий (ввода, кликов, изменения фильтров) в нижележащие слои через декларативную систему сигналов и слотов Qt.

Архитектурная топология

Взаимодействие графических компонентов приложения спроектировано по иерархическому принципу с обеспечением слабой связанности (Loose Coupling) и строгим разделением ответственности:

1. **Главное окно** (`MainWindow`) инициализирует и позиционирует на окне панель управления (`FilterPanel`) и табличное представление (`QTableView`). Также перехватывает сигналы от графических элементов и транслирует их в вызов методов модели данных (`MoexTableModel`).
2. **Панель фильтрации** (`FilterPanel`) - графический виджет ввода параметров. Компонент изолирован от табличного представления и структуры базы данных. Связь осуществляется через отправку сигналов Qt. При изменении параметров фильтрации панель генерирует сигнал `apply_filters_requested(FilterCriteria)`, передавая пакет объектов передачи данных (DTO-пакет) главному окну.
3. **Интерфейс визуализации** (`QTableView`) делегирует отрисовку строк кастомной табличной модели. Компонент не хранит состояние данных в своей памяти. таблица отрисовывает строки на лету только то, что видно на экране. Программа забирает минимум оперативной памяти, а само чтение данных из модели полностью защищено от сбоев и конфликтов в потоках
4. **Адаптер данных** (`MoexTableModel`) служит связующим звеном между графическим представлением `QTableView` и внутренним аналитическим движком на базе `pandas.DataFrame`. Модель принимает сигналы фильтрации, пересчитывает индексы строк и инициирует перерисовку ячеек в `QTableView`.

Панель фильтрации (`FilterPanel`)

Компонент `FilterPanel` (наследует `QWidget`) реализует визуальную область настройки параметров фильтрации.

- **Объект передачи данных (DTO)** - класс данных `FilterCriteria` на базе декоратора `@dataclass(frozen=True)` для передачи параметров фильтрации. DTO гарантирует, что данные фильтров передаются между слоем графического интерфейса и бизнес-логикой в неизменяемом, строго типизированном виде, исключая непреднамеренные изменения в пути.
- **Декларативное API сигналов** обеспечивает изоляцию панели от модели данных и внешних сервисов и общается с внешним миром только через четыре типизированных сигнала Qt:
 - `mode_changed = Signal(str)` - изменение режима отображения колонок;
 - `export_requested = Signal(str)` - запрос на экспорт;
 - `refresh_api_requested = Signal()` - запуск загрузки данных с биржи;
 - `apply_filters_requested = Signal(FilterCriteria)` - отправка DTO.
- **Метод `init_ui()`** собирает интерфейс на базе вложенных контейнеров (`QVBoxLayout`, `QHBoxLayout`, `QFrame`). Метод задает ограничения на ввод (например, `self.txt_ticker.setMaxLength(5)` для тикеров) и связывает события виджетов с внутренними защищенными слотами (например, клик по кнопке сброса с `self._on_reset_clicked`).

Главное окно (`MainWindow`)

Класс `MainWindow` (наследует `QMainWindow`) выступает главным окном графического приложения. Его главная задача заключается в том, чтобы подготовить внутренние сервисы к работе, собрать воедино внешний вид приложения и настроить гибкие связи между элементами, не связывая их друг с другом намертво.

- **Связывание компонентов при запуске** происходит в методе `__init__`, где сначала создаются объекты `FilterPanel`, `QTableView` и `MoexTableModel`, а затем настраивается передача сигналов между ними.

```
self.filter_panel.apply_filters_requested.connect(self._on_apply_filters)
self.filter_panel.refresh_api_requested.connect(self._on_refresh_data)
self.filter_panel.mode_changed.connect(self._on_display_mode_changed)
self.filter_panel.export_requested.connect(self._on_export_requested)
```

- **Слот асинхронного обновления** `_on_refresh_data()` управляет визуальным состоянием процесса сетевой загрузки. Перед началом сетевого запроса метод переводит интерфейс в состояние загрузки, вызывает `QCoreApplication.processEvents()` для мгновенной перерисовки UI.
- **Слот применения фильтров** `_on_apply_filters(self, criteria: FilterCriteria)` координирует процесс фильтрации данных. При получении пустого DTO метод отключает сортировку в `QTableView`, сбрасывает визуальный индикатор сортировки и состояние модели. Если переданы правильные параметры фильтрации, метод делегирует фильтрацию выделенной бизнес-службе `DataFilterService.filter_market_data`.
- **Оптимизация отрисовки при фильтрации** в методе `_on_apply_filters` реализована через умное обновление структуры, что позволяет избежать вызова тяжелого метода сброса модели `beginResetModel` / `endResetModel`. Если количество строк и индексы старой и новой выборок совпадают, модель обновляет только изменившиеся ячейки.

3.2. Кастомный виджет `NullableDoubleSpinBox`

При проектировании аналитических систем для финансовых рынков возникает довольно таки неприятная проблема обработки граничных состояний на стыке интерфейса и математического ядра. Стандартные числовые поля ввода Qt (`QDoubleSpinBox`) имеют фундаментальное ограничение: они обязаны всегда содержать валидное вещественное число в рамках заданного диапазона (например, от `0.0` до `10000000.0`).

Однако на бирже понятия «**математический ноль**» (`0.0`) и «**отсутствие данных**» (`None` / `NaN`) являются совершенно разными бизнес-состояниями. Если наивно использовать стандартный виджет, пользователь не сможет отключить фильтр по цене или объему, так как поле ввода будет принудительно подставлять числовое значение, заданное по умолчанию, что приведет к непреднамеренно неправильной фильтрации выборки `pandas.DataFrame`.

Для решения этой проблемы в модуле `filter_panel.py` был спроектирован кастомный виджет `NullableDoubleSpinBox`, расширяющий стандартный `QDoubleSpinBox` и реализующий концепцию проектирования интерфейса с защитой от ошибок.

Принцип единого источника истины (SSOT)

Вместо хранения ненадежных булевых флагов состояния активности, метод `is_empty()` динамически запрашивает состояние физического текстового поля внутри спинбокса.

```
def is_empty(self) -> bool:
    return not self.lineEdit().text().strip()
```

Текстовое поле выступает единственным источником правды о том, заполнено ли поле пользователем.

Подавление автоматической подстановки числа в `textFromValue()`

В стандартном Qt-виджете при установке значения `0.0` на экране принудительно отрисовывается `"0.00"`. Метод `textFromValue(self, value: float)` переопределен таким образом, чтобы анализировать фокус

ввода и текст.

```
def textFromValue(self, value: float) -> str:
    if not self.hasFocus() and not super().lineEdit().text().strip():
        return ""
    return super().textFromValue(value)
```

Если у поля нет фокуса и строка пуста, виджет возвращает пустую строковую маску "", подавляя нежелательную автоподстановку нулевых значений для пользователя.

Интерактивный перехват шага в `stepBy()`

Если поле было визуально пустым (состояние `None`), а пользователь кликнул по стрелочкам инкремента/декремента, стандартный виджет не среагирует корректно. Переопределенный метод `stepBy(self, steps: int)` решает эту проблему.

```
def stepBy(self, steps: int) -> None:
    if self.is_empty():
        self.setValue(0.0)
    super().stepBy(steps)
```

При первом физическом воздействии виджет мгновенно инициализирует значение до `0.0`, выводя его на экран, а затем применяет математический шаг изменения.

Связь с бизнес-логикой через `value()`

Метод `value()` переопределен для возврата полиморфного типа `float | None`.

```
def value(self) -> float | None:
    if self.is_empty():
        return None
    return super().value()
```

Верхние слои приложения не обязаны знать о внутреннем устройстве графического элемента. Слой управления (`MainWindow`) просто вызывает метод `.value()`, получая либо число, либо чистый `None`, который затем упаковывается в DTO `FilterCriteria` и передается в вычислительный модуль.

4. Конвейер данных и бизнес-логика скринера

Основная архитектурная задача бэкэнд-слоя программного комплекса - построение сквозного конвейера данных (End-to-End Data Pipeline). Такой подход автоматизирует путь информации от API до экрана без ручного вмешательства, обеспечивая единый источник истины и высокую наблюдаемость всей системы. На входе конвейер принимает сырой, неструктурированный поток сетевых пакетов от внешних серверов биржи, а на выходе формирует строго типизированную, валидированную и очищенную от технического шума матрицу в оперативной памяти компьютера.

4.1. Клиент связи (`client.py`)

Сетевое взаимодействие с программным интерфейсом Московской Биржи инкапсулировано в изолированном классе транспортного уровня `MoexClient`. Класс отвечает за диспетчеризацию HTTP-запросов к эндпоинту MOEX ISS API (рынок акций, режим основных торгов T+: `securities.json`), первичную валидацию входящих JSON-пакетов и абстрагирование сетевых протоколов от бизнес-логики приложения. Конвейер автоматически преобразует исходные сетевые данные от биржи в очищенную от лишней информации и готовую к вычислениям таблицу.

Асинхронно-ориентированный стек (`httpx`)

Вместо синхронной библиотеки `requests`, в качестве сетевого движка выбран HTTP-клиент `httpx`. Данное решение обеспечивает:

- строгий контроль сессий и обработку таймаутов от зависания потоков исполнения при нестабильном соединении;
- текущая структура клиента обеспечивает его быструю интеграцию с асинхронной моделью выполнения (`async/await`) без необходимости рефакторинга кода.

Архитектурный паттерн изоляции сбоев (`MoexAPIError`)

Чтобы слои приложения не зависели друг от друга, а интерфейс не падал, в модуле настроена защита от сбоев. Все сетевые проблемы - от обрыва интернета до ошибок сервера и таймаутов библиотеки `httpx` - перехватываются внутри методов загрузки (`fetch_from_api()`) и парсинга данных (`validate_and_parse()`).

Система логирования `loguru` сохраняет полную историю сбоя, а сама программа переводит техническую ошибку сети в понятное кастомное исключение `MoexAPIError`. Благодаря этому графическое окно `MainWindow` получает информацию о проблемах с биржей, вообще не зная, как именно устроена сеть под капотом.

Механизм автономной отладки и система тестовых окружений

Внутри метода конвейера `run_pipeline(use_fixture: str | None)` реализована двухканальная архитектура поставки данных. При передаче строкового идентификатора в параметр `use_fixture`, клиент мгновенно переключается в изолированный офлайн-режим, выполняя чтение локального слепка данных через метод `fetch_from_fixture()`. Это решает три инженерные задачи:

- **отказоустойчивость разработки** гарантирует возможность проектирования и отладки компонентов интерфейса при отсутствии сети или во время технических работ на стороне биржи;
- **стабильность тестирования** достигается путем обеспечения полной воспроизводимости проверок с использованием неизменяемых эталонных JSON-фикстур;
- **снижение нагрузки** минимизирует количество избыточных запросов к серверам биржи в процессе циклов локальной отладки.

4.2. Схемы типов данных и метаданных биржи (`extractor.py`)

Ответы MOEX ISS API возвращаются в виде JSON-ответов, где данные и метаданные разделены. Блоки `securities` (справочный реестр) и `marketdata` (оперативные котировки) содержат независимые вложенные словари `metadata`, описывающие физические типы колонок.

Для обработки этой специфики спроектирован выделенный статический сервис `MoexSchemaExtractor`.

Принцип единственной ответственности (SRP)

В соответствии с SOLID-принципами программной инженерии, задача анализа структуры типов полностью вынесена за пределы основного конвейера парсинга. Класс `MoexDataParser` не занимается угадыванием типов или ручным приведением конкретных полей. Вместо этого он делегирует инспекцию сырого ответа классу `MoexSchemaExtractor`.

Динамическая инспекция типов

Классической уязвимостью финансовых систем является завязка на жестко прописанные типы данных (например, когда разработчик жестко указывает в коде: колонки `LAST` и `VOLUME` - это `float`, а `SECID` - `str`). При изменении спецификации API или добавлении биржей новых расчетных показателей такие системы аварийно завершают работу.

В `MoexSchemaExtractor.extract_numeric_columns(raw_data)` реализован алгоритм динамического сканирования схем данных на месте. Метод последовательно итерирует по корневым табличным узлам `securities` и `marketdata`. Извлекает структуру `metadata` и анализирует строковые спецификации типов, задекларированные ядром биржи (поиск подстрок `double`, `float`, `int32`, `int64`). На основе сигнатур динамически формирует хеш-множество `numeric_fields: set[str]`, содержащее только системные имена столбцов.

```
for col_name, info in metadata.items():
    data_type = info.get("type", "").lower()
    if "double" in data_type or "int" in data_type or "float" in data_type:
        numeric_fields.add(col_name)
```

Это решение гарантирует **адаптивность конвейера данных**. Если биржа изменит точность какого-либо показателя (например, переведет целочисленный объем торгов в формат `double`), приложение автоматически подстроится под изменения.

4.3. Объединение и трансформация данных (`parser.py`)

Для преобразования JSON-структур, полученных от сетевого клиента, в итоговые таблицы был спроектирован класс `MoexDataParser`. Главная задача данного компонента - преодоление структурных ограничений протокола MOEX ISS API и нормализация данных перед их передачей в интерфейсную модель.

Механизм денормализации

Специфика архитектуры Московской Биржи заключается в том, что оперативная информация об одном и том же инструменте разделена на два изолированных информационных блока:

- `securities` - статический справочник (краткое и полное наименование компании, ISIN, уровень листинга и т.д.);
- `marketdata` - динамический биржевой стакан (последняя цена сделки `LAST`, объемы торгов `VOLTODAY`, ценовые изменения и т.д.).

Для создания общей таблицы метод `parse_to_dataframe(raw_data)` объединяет данные в оперативной памяти по принципу `SQL LEFT JOIN`, используя быстрые инструменты библиотеки `pandas`. Процесс проходит в три последовательных этапа.

1. Метод `_extract_block(raw_data, block_key)` безопасно извлекает блоки `securities` и `marketdata`, превращая их в отдельные промежуточные таблицы `pandas.DataFrame`.
2. Программа автоматически находит числовые колонки с помощью `MoexSchemaExtractor.extract_numeric_columns` и приводит их к типу `float`. Это защищает код от частой ошибки, когда цифры случайно считываются как обычный текст.
3. Финальное объединение таблиц происходит через функцию `pandas.merge(..., how="left", on="SECID")`. В качестве главного связующего ключа система использует уникальный тикер акции `SECID`.

Это все позволяет собрать единый плоский массив данных, готовый к фильтрации.

Очистка от технических колонок

Ответ серверов MOEX ISS API избыточен и содержит более 50 системных колонок (технические маркеры, временные метки серверов, неиспользуемые статусы торгов), большая часть которых не имеет бизнес-ценности для пользователя. Накопление таких данных в ОЗУ приводит к расходу памяти и замедлению работы графики.

Для оптимизации потребления ресурсов в парсере спроектирован защищенный метод `_filter_columns(df)`. Метод работает по принципу "разрешающего списка". Он оставляет в таблице только те столбцы, которые требуются текущим режимом конфигурации интерфейса. Перед этим алгоритм проверяет, действительно ли нужные поля есть в ответе биржи. Это позволяет избежать падения программы с ошибкой `KeyError`, а в случае отсутствия поля система отправляет предупреждение в `logger.warning`.

Затем метод принудительно ставит уникальный тикер `SECID` на самую первую позицию в таблице для правильной связи с графическим окном. На финальном этапе все остальные неиспользуемые столбцы удаляются из памяти, мгновенно освобождая ОЗУ компьютера.

4.4. Аналитический движок векторизованной фильтрации (`analytics.py`)

Вычислительное ядро приложения инкапсулировано в классе `DataFilterService`. Его ключевая функция - обеспечение интерактивной фильтрации рыночной матрицы в реальном времени по запросу от графической панели управления `FilterPanel`.

Для достижения максимального быстродействия в методе `filter_market_data` реализована векторизация на уровне булевых масок. Вместо построчного перебора этот алгоритм мгновенно обрабатывает колонки целиком с помощью быстрых математических операций библиотеки Pandas в три последовательных шага.

1. **Инициализация единого вектора правды** происходит в самом начале вычислений, когда в оперативной памяти выделяется один массив булевых значений. Его начальное состояние содержит только элементы `True`, что означает изначальную валидность всех строк.
2. **Побитовые модификации с константными затратами памяти** позволяют избежать физического усечения или копирования тяжелой таблицы. Каждый шаг фильтрации просто накладывает условия на булев вектор с помощью быстрых побитовых операций И. Например, при фильтрации по тикеру применяется условие `mask &= df[col_secid].str.contains(ticker)`, а при поиске по диапазону цен вектор обновляется выражением `mask &= df[col_last].notna() & (df[col_last] >= price_from)`.
3. **Единоразовое ленивое копирование** завершает конвейер фильтрации. Физическое усечение исходной таблицы и создание результирующего среза происходят ровно один раз в самом конце метода по формуле `return df[mask].copy()`.

Такой подход гарантирует, что временная сложность по выделению дополнительной памяти падает до константного уровня $O(1)$ относительно количества критериев фильтрации.

4.5. Конфигурация и константы системы (`constants.py`, `config.py`)

Удобство и безопасность разработки обеспечивается за счет выноса декларативных правил системы в изолированные конфигурационные компоненты.

Исключение магических строк (`MoexColumns`)

Взаимодействие со сторонними API часто сопряжено с рисками опечаток в строковых ключах. Ошибки вроде написания `SEC_ID` вместо `SECID` не обнаруживаются линтерами и проявляются только на этапе выполнения в виде падений приложения.

Для защиты от этих сбоев в модуле `constants.py` был спроектирован строгий типизированный класс перечислений `MoexColumns(str, Enum)`. Он выступает в качестве единого источника истины. Все системные имена полей, включая `SECID`, `LAST` и `VOLTODAY`, жестко зафиксированы внутри этого класса.

Использование конструкции `MoexColumns.LAST.value` гарантирует два важных инженерных преимущества. Во-первых, любая опечатка разработчика мгновенно перехватывается статическим анализатором `Ruff` или линтером среды разработки еще на этапе написания кода. Во-вторых, при изменении спецификации API на стороне Московской Биржи инженеру достаточно обновить всего одну строку в файле `constants.py`.

Режимы отображения таблиц `UIConfig`

Для управления видимостью колонок в зависимости от уровня подготовки пользователя в модуле `config.py` разработан класс `UIConfig`. Он полностью инкапсулирует логику переключения режимов отображения.

Важным элементом конфигуратора является алгоритм метода `get_columns_to_show(all_available_columns)`. Он эффективно решает проблему динамических интерфейсов. Если в коде закрепить список колонок, а биржа в конкретный день не пришлет одну из них, таблица `QTableView` отобразит пустой белый столбец. Чтобы избежать этого, алгоритм выполняет пересечение множеств на лету.

```
if self.current_mode == "basic":
    return [col for col in DEFAULT_COLUMNS if col in all_available_columns]
```

Метод сопоставляет выбранный пользователем готовый набор столбцов со списком полей, фактически присутствующих в текущем ответе биржи. В результате интерфейс получает только валидный и очищенный список, что гарантирует стопроцентную заполненность каждого видимого столбца на экране.

5. Обеспечение качества кода

Программа должна работать автономно и стабильно на конечных устройствах пользователей. Этот раздел посвящен качеству кода и инфраструктуре тестирования. В нем рассматриваются механизмы наблюдения за состоянием системы и статический анализ с помощью инструмента `Ruff` для контроля чистоты кода и соблюдения стандартов PEP. Отдельное внимание уделено архитектуре тестового пакета на базе `pytest` и `pytest-qt` с применением изоляции через фикстуры и локальные JSON-заглушки для проверки бизнес-логики без обращения к реальному API биржи.

5.1. Инфраструктура логирования (`loguru`)

Для непрерывного мониторинга состояния системы, облегчения процесса отладки и оперативного расследования инцидентов (Post-mortem Analysis) в проекте развернута централизованная подсистема логирования на базе современной асинхронно-ориентированной библиотеки `loguru`. Настройка и инициализация инфраструктуры вынесены в отдельный модуль `logger_config.py`.

Многоканальное логирование (Sinks)

Вместо стандартного вывода всех сообщений в одну кучу, подсистема логов разделена на два независимых и изолированных канала.

Первый канал представляет собой интерактивный отладочный поток для вывода в консоль через стандартный `sys.stdout`. Чтобы локальная разработка была удобной, здесь настроена цветная разметка сообщений и подробный формат строки с указанием времени, модуля, функции и конкретной строки кода. Данный канал работает на самом детальном уровне фильтрации `DEBUG`, что позволяет разработчику видеть полную картину происходящего в терминале среды разработки во время отладки.

Второй канал - это постоянный файловый поток, который ведет запись в физический файл `moex_screener.log` в специальной папке внутри проекта. Чтобы сэкономить место на диске и убрать лишний технический шум, этот канал настроен на более строгий уровень `INFO`. В файл записываются

только ключевые этапы жизни приложения, включая успешный запуск, завершение конвейера данных, экспорт файлов и предупреждения о сетевых сбоях.

Автоматическое обслуживание и очистка логов (Retention & Rotation)

Бесконтрольная запись логов в настольных приложениях часто приводит к переполнению жестких дисков пользователей и истощению дискового пространства. Для предотвращения этой проблемы в конфигурации логгера были реализованы три политики защиты.

1. Первая политика отвечает за автоматическую ротацию файлов объемом до пяти мегабайт. Как только активный файл лога превышает этот размер, подсистема логирования закрывает его, переименовывает в архивный файл с добавлением точной временной метки и мгновенно создает новый чистый документ для продолжения записи.
2. Вторая политика обеспечивает автоматическое сжатие полученных архивов в формат ZIP силами встроенного движка библиотеки `loguru`. Данный шаг позволяет эффективно уменьшить физический объем архивных файлов на жестком диске более чем на 80–90%.
3. Третья политика устанавливает правила удержания данных за последние семь дней. Подсистема постоянно сканирует рабочую папку с логами и безвозвратно удаляет с диска все архивные файлы, которые были созданы более недели назад.

Ниже представлен детальный, технически насыщенный текст для **Раздела 5 «Обеспечение качества кода (Quality Assurance & Test Design)»** твоего отчета.

5.2. Статический анализ кода (`Ruff`)

Современные стандарты промышленной разработки программного обеспечения требуют автоматизированного контроля качества исходного кода на всех этапах его жизненного цикла. Для предотвращения накопления технического долга, унификации стиля кодирования в соответствии со стандартами PEP 8 и пресечения появления потенциальных уязвимостей в проекте развернут современный инструмент статического анализа - `Ruff`.

Обоснование выбора

В отличие от классической связки утилит (например, `Flake8` для поиска синтаксических ошибок, `Black` для форматирования и `isort` для сортировки импортов), в качестве единого решения был выбран `Ruff`. Написанный на языке Rust, он превосходит альтернативные инструменты на Python по скорости анализа в десятки и сотни раз, выполняя проверку кодовой базы практически мгновенно ($O(1)$ относительно времени сборки коммитов).

Конфигурация в `pyproject.toml`

Параметры автоматической проверки жестко зафиксированы в конфигурационном файле проекта. Настройки указывают анализатору ориентироваться на синтаксис и оптимизацию под возможности современных стандартов языка. Ограничение длины строки фиксирует строгий лимит, что соответствует общепринятым соглашениям форматирования и предотвращает появление горизонтальной прокрутки в редакторе.

Эти правила автоматической проверки решают три задачи для поддержания чистоты в коде.

1. Первая группа правил отвечает за мгновенное выявление ошибок стиля и синтаксических аномалий, включая неиспользуемые элементы и проблемы с подключением модулей.
2. Вторая категория обеспечивает автоматический контроль порядка и группировки внешних библиотек.
3. Третья группа выдает рекомендации по обновлению синтаксиса, предлагая автоматически заменить устаревшие конструкции на современные нативные аналоги.

Регламент

В проекте реализован строгий регламент контроля качества перед фиксацией изменений в репозитории. Каждый обязан выполнять локальную валидацию с помощью утилит командной строки перед созданием коммита.

Для этого сначала запускается команда `poetry run ruff format .` для автоматического выравнивания отступов, переносов и приведения структуры файлов к единому стилю. Сразу после этого применяется команда `poetry run ruff check . --fix` для глубокого сканирования кодовой базы. Этот шаг позволяет не только обнаружить проблемы, но и устранить простые нарушения, например, мгновенно удалить неиспользуемые импорты и переменные.

5.3. Автоматизация тестирования (`pytest` и `pytest-qt`)

Для обеспечения непрерывной интеграции (CI) и исключения регрессионных ошибок при доработке аналитических алгоритмов спроектирована иерархическая структура тестового покрытия, базирующаяся на фреймворке `pytest`.

Структура тестового каталога (`tests/`)

Тестовый пакет полностью изолирован от основного кода приложения и организован в виде отдельных тестовых модулей внутри специальной директории.

```
tests/
├── conftest.py           # Глобальные фикстуры тестов
├── test_analytics.py     # Тестирование математических формул и аналитических расчётов
├── test_api.py           # Тесты сетевого клиента
├── test_filter_panel.py  # Изолированные тесты элементов UI панели управления
├── test_main_window.py  # Интеграционные тесты главного окна
├── test_model.py         # Тестирование поведения таблицы MoexTableModel
├── test_parser.py        # Тестирование парсера
└── test_ui.py           # Базовые смюк-тесты отрисовки графических компонентов
```

Тестирование GUI (`pytest-qt`)

Тестирование графических интерфейсов представляет собой сложную задачу из-за того, что нативные виджеты сильно завязаны на глобальный цикл обработки событий операционной системы. Обычный вызов методов интерфейса в тестах приводит к их блокировке или непредсказуемому поведению. Для решения этой проблемы была внедрена специализированная библиотека `pytest-qt`, которая предоставляет управляющий инструмент в виде фикстуры `qtboto`.

Пример сценария тестирования панели фильтрации через этот инструмент эффективно решает две важные задачи.

Первая задача заключается в эмуляции физического ввода пользователя. Команда отправки клавиш `qtboto.keyClicks(widget, "SBER")` физически посылает сигналы нажатия в текстовое поле ввода тикера, заставляя внутренние механизмы графического движка обрабатывать в реальном масштабе времени.

Вторая задача обеспечивает проверку асинхронных сигналов интерфейса. С помощью контекстного менеджера `with qtboto.waitSignal(panel.apply_filters_requested, timeout=1000) as blocker` тест приостанавливает выполнение до тех пор, пока виджет не сгенерирует целевой сигнал в ответ на клик по кнопке. Это позволяет надёжно проверять корректность формирования и передачи неизменяемых DTO-объектов критериев фильтрации без запуска всего программного комплекса целиком.

5.3. Изоляция тестов и мокирование

Критическим требованием к юнит-тестам является их **детерминированность** (тест должен выполняться одинаково успешно на любой машине, в любое время, без интернета) и скорость работы. Обращение к реальному API биржи во время прогона тестов нецелесообразно, так как:

- Биржа может проводить технические работы (сетевой таймаут спровоцирует ложный сбой теста).
- На серверах MOEX котировки непрерывно меняются - невозможно написать тест с жесткими эталонными проверками (Assertion), если данные на входе динамичны.
- Скорость выполнения сетевых запросов замедляет прогон тестов до нескольких секунд вместо миллисекунд.

Для тестирования парсера `MoexDataParser` и аналитического сервиса `DataFilterService` в проекте используется каталог фикстур, содержащий реальные слепки ответов от MOEX ISS API в формате статических JSON-файлов.

Файл `conftest.py` содержит переиспользуемые генераторы состояний - фикстуры `pytest`. Пример фикстуры, загружающей локальный слепок данных для тестирования:

```
import json
import pytest
from pathlib import Path

@pytest.fixture(scope="session")
def raw_moex_response():
    """Загружает статическую фикстуру биржи из локального хранилища."""
    fixture_path = Path(__file__).parent / "fixtures" / "securities_response.json"
    with open(fixture_path, "r", encoding="utf-8") as f:
        return json.load(f)
```

Для тестирования транспортного клиента `MoexClient` применяется подмена сетевого контекста с использованием библиотеки `pytest-mock`. Вместо отправки реального HTTP-запроса через библиотеку `httpx`, вызов сетевого сокета перехватывается, и вместо него возвращается предопределенный локальный объект:

```
def test_fetch_from_api_success(mocked, raw_moex_response):
    # Паттерн Mock: перехватываем отправку GET-запроса в httpx.Client
    mock_get = mocked.patch("httpx.Client.get")

    # Конфигурируем поведение заглушки: возвращать статус 200 и наш JSON
    mock_response = mocked.Mock()
    mock_response.status_code = 200
    mock_response.json.return_value = raw_moex_response
    mock_get.return_value = mock_response

    client = MoexClient()
    result = client.fetch_from_api()

    # Проверяем, что метод вернул валидный JSON и не совершал реальных сетевых подключений
    assert result == raw_moex_response
    mock_get.assert_called_once()
```

Благодаря изоляции тестов от внешней среды весь тестовый пакет приложения выполняется с высокой скоростью, гарантируя высокую стабильность контроля качества.

6. Инфраструктура и дистрибуция

6.1. Управление зависимостями (`Poetry`)

Одной из классических проблем при разработке программного обеспечения на языке Python является деградация зависимостей («dependency hell»). Использование стандартного инструмента `pip` совместно с файлом `requirements.txt` не гарантирует воспроизводимость сборки на машинах других разработчиков или серверах непрерывной интеграции (CI/CD), так как транзитивные (вложенные) зависимости библиотек могут обновляться независимо, приводя к непредсказуемым runtime-ошибкам.

Для обеспечения абсолютной детерминированности среды выполнения в проекте развернут менеджер пакетов нового поколения - **Poetry**.

Манифест `pyproject.toml`

Вместо хранения разрозненных конфигурационных файлов для линтеров, тестов и зависимостей, менеджер пакетов Poetry объединяет все метаданные проекта в едином стандартизированном манифесте `pyproject.toml`. Все зависимости четко разделены на две изолированные группы.

Основное окружение содержит библиотеки, необходимые для сборки и стабильной работы бизнес-логики приложения. Сюда входят графический движок PySide6, библиотека Pandas для обработки таблиц и сетевой клиент httpx. Привязка версий осуществляется с использованием специальных символов тильды или каретки. Это гибко ограничивает мажорные обновления, способные нарушить обратную совместимость кодовой базы.

Окружение разработки изолирует утилиты, требующиеся только на этапе написания кода и тестирования. В эту группу вынесены тестовые пакеты pytest и pytest-qt, статический анализатор ruff, а также инструмент сборки pyinstaller. Такое разделение гарантирует, что конечный дистрибутив программы не будет перегружен избыточными сторонними библиотеками.

Файла блокировки `poetry.lock`

Ключевым инженерным решением для достижения стопроцентной воспроизводимости сборки является автоматическая генерация и версионирование файла блокировки `poetry.lock`. Его работа строится на трех последовательных механизмах.

Разрешение конфликтов происходит при выполнении команды установки пакетов, когда утилита автоматически выстраивает полный граф зависимостей и выявляет взаимные требования всех библиотек.

Фиксация версий гарантирует, что в лок-файл записываются не просто диапазоны, а конкретные точные версии вплоть до минорного патча для каждой установленной библиотеки и всех её внутренних зависимостей.

Контроль целостности обеспечивает дополнительную безопасность, сохраняя криптографические хеш-суммы для каждого пакета. При развертывании проекта на новой машине менеджер пакетов сверяет хеши скачиваемых файлов с заблокированными, полностью исключая подмену данных в сети и гарантируя, что сторонний код не был модифицирован.

Использование связки манифеста и файла блокировки гарантирует, что любой разработчик в команде или автоматический сервер сборки развернет абсолютно идентичное программное окружение байт-в-байт.

Использование связки `pyproject.toml` + `poetry.lock` гарантирует, что любой разработчик команды или автоматический CI-сервер развернет абсолютно идентичное окружение байт-в-байт.

6.2. Шаблионизация модулей (`Copier`)

Для автоматизации развертывания новых программных компонентов и поддержания единой структуры каталогов в проекте применен инструмент Copier.

Стандартизация структуры проекта достигается за счет того, что Copier генерирует новые файлы на основе декларативных шаблонов Jinja2. Это полностью исключает ручное создание папок для исходного кода, тестов и базовых конфигурационных файлов при добавлении новых аналитических модулей.

Контекстный рендеринг через конфигурационный файл утилиты позволяет проводить интерактивный опрос разработчика в командной строке. На основе этих ответов система динамически подставляет в шаблоны актуальные метаданные, включая имя автора, версию Python и контактную электронную почту.

Бесконфликтные обновления инфраструктуры стали возможны благодаря механизму трехстороннего слияния, который выгодно отличает этот инструмент от классических генераторов кода. Данная технология позволяет безопасно обновлять общую конфигурацию проекта, например, новые правила линтера в общем манифесте или шаблоны сборщика, с помощью простой команды обновления без риска затереть написанную разработчиком бизнес-логику.

6.3. Сборка бинарных артефактов (PyInstaller)

Пользователи финансовых скринеров не обязаны иметь установленный интерпретатор Python, менеджер пакетов Poetry или сторонние библиотеки. Был спроектирован конвейер компиляции исходного кода в автономный файл с расширением `exe` с помощью утилиты `PyInstaller`.

Сборка в один файл объединяет байт-код, динамические библиотеки, ресурсы графического интерфейса и сам интерпретатор в единый сжатый артефакт. При запуске этот файл автоматически распаковывает свое содержимое во временную директорию, обеспечивая работу программы без предварительной установки.

Флаг отключения консоли переводит приложение в оконный режим операционной системы Windows, что скрывает черное окно командной строки и улучшает пользовательский опыт.

Параметр добавления данных декларативно упаковывает локальные JSON-заглушки внутрь бинарного контейнера, гарантируя программе локальный доступ к статическим данным для работы в офлайн-режиме. Внедрение этого конвейера дистрибуции позволяет получить компактный автономный файл, готовый к мгновенному запуску на любом целевом компьютере.

7. Заключение и управление техническим долгом (Technical Debt)

7.1. Итоги разработки MVP

В ходе прохождения производственной практики в ООО «ПРОКСИ» (г. Курск) был успешно спроектирован, реализован и протестирован минимально жизнеспособный продукт (MVP) - кроссплатформенный настольный скринер акций Московской Биржи («Baika»).

Приложение построено на базе современной библиотеки `PySide6 (Qt)` для отображения графического интерфейса, библиотеки `pandas` для высокоскоростной трансформации табличных данных и асинхронно-ориентированного HTTP-клиента `httpx` для связи с серверами MOEX ISS API.

В ходе выполнения первой итерации разработки успешно завершено проектирование и развертывание базовой версии минимально жизнеспособного продукта. Все запланированные технические и архитектурные показатели выполнены в полном объеме, а созданная программная инфраструктура полностью готова к дальнейшему масштабированию и передаче в опытную эксплуатацию.

Разработанный программный комплекс спроектирован с разделением функциональных слоев, что исключает появление жестких зависимостей в кодовой базе и позволяет развивать интерфейсную и расчетную логику независимо друг от друга. Написанный код полностью соответствует отраслевому

стандарту PEP 8 и автоматически проверяется при каждой фиксации изменений, что гарантирует высокое качество кодовой базы при командной работе.

Созданный сквозной конвейер данных полностью автоматизирует путь информации от серверов Московской Биржи до экрана пользователя, сохраняя при этом высокую отзывчивость интерфейса и стабильная кадровая чистота за счет выноса тяжелых вычислений в фоновые потоки. Защитные механизмы и кастомная система обработки ошибок надежно защищают приложение от сетевых сбоев и платформенных аномалий операционной системы Windows, обеспечивая стабильный старт программы на любых устройствах.

Инфраструктура тестирования и автоматическая ротация ресурсов позволяют проводить автономную отладку и гарантируют предсказуемый объем занимаемой памяти. Итоговая сборка проекта автоматизирована и на выходе выдает автономный файл, готовый к запуску пользователями без предварительной настройки окружения.

7.2. Дальнейшие направления развития проекта

Несмотря на работающее состояние текущей версии проекта, архитектура MVP содержит ряд сознательных компромиссов и технических долгов, которые должны быть исправлены или же быть улучшены в рамках последующей работы над проектом.

Многопоточность (Worker-Object)

В текущей реализации MVP сетевой запрос к MOEX ISS API выполняется синхронно. Несмотря на использование эффективных таймаутов в клиенте `MoexClient`, выполнение сетевых операций ввода-вывода непосредственно в основном потоке графического интерфейса GUI является недостатком. Во время ожидания ответа от серверов биржи (время задержки может достигать нескольких секунд при нестабильном соединении) интерфейс приложения полностью «замораживается», переставая реагировать на действия пользователя и системные события операционной системы.

Для обеспечения плавного и отзывчивого интерфейса в следующей итерации разработки будет внедрена многопоточная архитектура на основе паттерна исполнения в рабочем объекте (`workers`). Это позволит изолировать графическую оболочку приложения от ресурсоемких вычислительных процессов и сетевых задержек.

В рамках реализации этого подхода вся бизнес-логика сетевого взаимодействия и парсинга данных будет скрыта внутри отдельного класса-исполнителя. Для его работы в системе создается выделенный поток, в который данный исполнитель принудительно переносится специальным системным вызовом.

Взаимодействие между главным потоком графического интерфейса и фоновым рабочим потоком будет организовано через механизм сигналов и слотов с использованием очередей сообщений. По этой схеме сигнал обновления из интерфейса запускает выполнение конвейера в фоне, а результирующий сигнал готовности данных возвращает обработанную таблицу для безопасного обновления модели отображения.

Разделение по функциональным возможностям (Feature-First)

Текущая файловая структура проекта организована по классическому принципу разделения технических слоев, при котором файлы группируются по своему назначению в отдельные папки. При дальнейшем расширении возможностей приложения, включая добавление графиков, систем уведомлений или портфельного анализа, такая структура приведет к сильной пространственной разрозненности кода каждого отдельного новшества. Это существенно увеличит ментальную нагрузку на разработчиков и усложнит параллельную работу инженерной команды над проектом.

Запланирован полный рефакторинг архитектуры каталогов репозитория в соответствии с методологией группировки по функциональным возможностям. При таком подходе каждая крупная функция системы проектируется как независимый и слабо связанный модуль, содержащий внутри себя собственный интерфейс, логику и тесты. Это позволит при необходимости легко отключать, изолированно тестировать или заменять отдельные компоненты программного комплекса без влияния на стабильность другой частей системы.

Автоматизация сборки (CI/CD)

Ручная компиляция исполняемых файлов с помощью инструмента сборки на локальной машине подвержена различным ошибкам и не позволяет создавать дистрибутивы под разные платформы в автоматическом режиме. Для устранения этих рисков и полной автоматизации процессов поставки в следующей итерации будет спроектирован конвейер непрерывной интеграции и развертывания на платформе автоматизации процессов разработки.

На этапе тестирования и контроля качества при каждом обновлении веток или открытии запросов на слияние облачный сервер будет разворачивать изолированное окружение с помощью менеджера пакетов. Пайплайн запустит глубокую проверку чистоты кодовой базы через статический анализатор и выполнит полный пакет модульных и интеграционных тестов.

На этапе автоматической сборки и выпуска релизов при фиксации новой версии проекта будет запускаться параллельная матричная компиляция на облачных виртуальных машинах под управлением Windows и macOS. Конвейер выполнит сборку автономных бинарных пакетов, упакует ресурсы и автоматически опубликует готовые релизные артефакты в специальный раздел репозитория. Данное решение должно исключить влияние человеческого фактора на качество сборки и гарантирует мгновенную доступность обновлений для конечных пользователей.